

ENGINEERING REBOOT 2026

Day 31 Field Manual

Version 1.0

Implementation Edition (Manuscript)

Mission Brief

Today marks the beginning of the Software Design Foundations phase of the Engineering Reboot.

Up to this point, programs have primarily been procedural, with data and functions existing separately. Today introduces the core idea behind object-oriented programming (OOP): **combining data and behavior into cohesive objects.**

The goal is not merely to learn the syntax of `class` and `__init__`, but to begin thinking about software as a collection of collaborating entities that model real-world concepts.

Learning Objectives

By the end of Day 31, you should be able to:

- Define Python classes.
 - Instantiate multiple objects.
 - Implement constructors using `__init__()`.
 - Store object state with instance attributes.
 - Create methods that operate on object state.
 - Recognize when a class is an appropriate abstraction.
-

Engineering Concepts

Today's engineering concepts include:

- Classes
 - Objects
 - Constructors
 - Instance attributes
 - Instance methods
 - `self`
 - Object lifecycle
 - Encapsulation (introduction)
-

Exercises

Exercise 1 — Your First Class

Objective

Create a simple class representing a person.

Suggested filename:

`person_demo.py`

Suggested tasks:

- Define a `Person` class.
 - Store a name and age.
 - Instantiate multiple `Person` objects.
 - Display their information.
-

Exercise 2 — Constructors

Suggested filename

`constructor_demo.py`

Practice:

- Implement `__init__()`.
- Observe how object initialization occurs.
- Compare constructor parameters with instance attributes.

Exercise 3 — Instance Methods

Suggested filename

`methods_demo.py`

Implement methods that:

- Display information.
- Modify object state.
- Demonstrate that each object maintains its own independent data.

Exercise 4 — Multiple Objects

Suggested filename

`multiple_objects_demo.py`

Create several objects and demonstrate:

- Independent state
- Independent behavior
- Method invocation on each object

Mini-Project

Employee Management System

Suggested filename

`employee_management.py`

Objective

Replace the dictionary-based representation of an employee with an object-oriented design.

Required Class

Employee

Suggested attributes:

- employee_id
 - first_name
 - last_name
 - department
 - position
 - salary
 - hire_year
-

Required Behaviors

Implement methods to:

- Display employee information.
 - Increase salary by a specified amount or percentage.
 - Transfer an employee to a different department.
 - Update the employee's position.
 - Calculate years of service.
 - Return the employee's full name.
-

Program Requirements

The program should:

- Create multiple `Employee` objects.
 - Store them in a list.
 - Iterate through the collection.
 - Invoke methods on each object.
 - Display formatted output.
 - Demonstrate that object state changes independently.
-

Stretch Goals (Optional)

If time permits:

- Add validation to prevent negative salaries.
 - Implement `__str__()` for readable object output.
 - Sort employees by salary or years of service.
 - Search employees by ID or last name.
-

Expected Deliverables

At the end of Day 31, the following should exist:

- Completed exercises
 - Working mini-project
 - Execution screenshots
 - Code screenshots
 - `Notes.md`
 - `Journal.md`
 - Git commit
 - Git push
 - Git operations evidence
-

Engineering Reflection

As you work through today's exercises, consider the following questions:

1. Why is an `Employee` object preferable to a dictionary for this application?
 2. Which data belongs inside the object, and which operations should be methods?
 3. How does encapsulating state and behavior improve maintainability?
 4. What advantages does this design provide if new employee features are added later?
-

Looking Ahead

Tomorrow you will extend today's work by exploring **methods and encapsulation** in greater depth. Rather than simply storing data, your objects will begin enforcing rules that protect their own internal state.